bighub

18BIG

Big Data Tools & Architecture

Lecture 04 — Delta Lake: a table format on top of Parquet

www.bighub.ai

Your guides to applied AI

Recap: last time

Three questions from **Session 03 — Storage Formats for the Lakehouse**:

1. In **columnar** storage, `SELECT sum(amount)` reads *what* off disk — and why is that fast?
2. The two kinds of **pushdown** — which comes from `SELECT`, which from `WHERE`, and which part of the file powers the `WHERE` one?
3. A folder of **Parquet** files is columnar and compressed — name the things it does **not** give you.

> Warm-up — answer out loud before we build on it.

Recap: answers

1. Only the **amount column chunk** — columnar keeps each column contiguous, so a narrow query reads ~6% of the bytes, not whole rows.
2. **Projection** ← `SELECT` (reads only chosen column chunks); **predicate** ← `WHERE` (skips row groups). Predicate pushdown is driven by the **footer** min/max statistics.
3. No **ACID transactions**, no reliable **upsert / MERGE**, no **history / time travel**, no safe **concurrent writes**. → that gap is today's lecture.

Today

- **Why files aren't a table** — what breaks without a log
- **Delta = Parquet + a transaction log** — the core idea
- **Inside `_delta_log`** — commits, actions, checkpoints
- **ACID** via optimistic concurrency
- **The operations** — append · **MERGE** upsert · time travel · schema evolution
- **Housekeeping** — OPTIMIZE & VACUUM
- **The landscape** — Delta vs Iceberg vs Hudi, catalogs, DuckLake
- **Hands-on** — build the first Silver Delta table

bighub

Part 1

Why files aren't a table

Your guides to applied AI

Parquet gave us a lot — but it's just files

What Parquet gives you

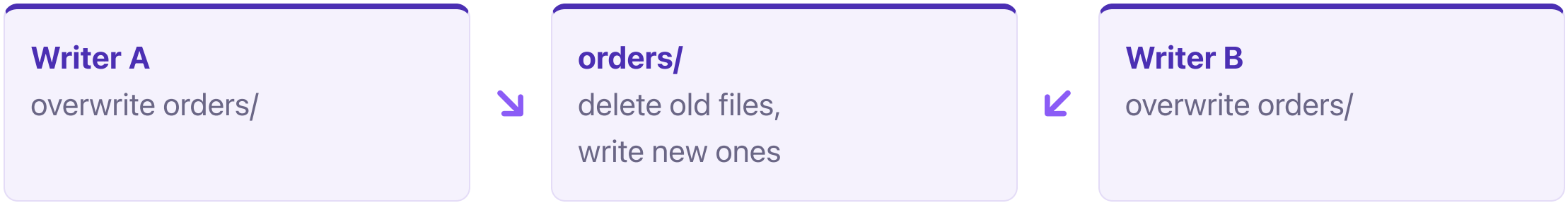
- Columnar, compressed, self-describing
- Pushdown + statistics
- Read by everything (DuckDB, Polars, Spark…)

What a *folder of files* can't do

- No **ACID transactions**
- No reliable **upsert / MERGE**
- No **history / time travel**
- No safe **concurrent writes**

> A directory of Parquet is a **snapshot**, not a table. The moment two people touch it, "which version am I reading?" has no answer.

What actually breaks



The failures

- **Overwrite isn't atomic** — a reader mid-write sees half the files
- **Two writers** clobber each other → silent data loss
- **No rollback** — the old state is gone the instant you overwrite

What we actually want

- A reader always sees **one consistent version**
- Writers that **don't corrupt** each other
- **Update / delete** single rows
- Keep **old versions** to audit & roll back

The fix: wrap the files in a log

Parquet
file format
the data

+

Transaction log
_delta_log/
what changed, when

=

Delta Lake
table format
ACID · MERGE · time travel

- The data stays **plain Parquet** — still read by everything.
- A **table format** adds an ordered log that says *which files make up the table right now*.
- **Delta · Iceberg · Hudi** are three takes on the same idea.

Same files on disk — the log is what turns them into a table

bighub

Part 2

Delta = Parquet + a transaction log

Your guides to applied AI

What's on disk

```
customers_delta/
  part-00000-....snappy.parquet                ← data (plain Parquet)
  part-00001-....snappy.parquet
  _delta_log/
    00000000000000000000.json                  ← commit 0  (protocol, metaData, add)
    00000000000000000001.json                  ← commit 1  (add, remove)
    00000000000000000002.json                  ← commit 2  (add)
    00000000000000000010.checkpoint.parquet    ← state snapshot (every 10 commits)
    _last_checkpoint                            ← pointer to the newest checkpoint
```

The `_delta_log/` folder — **append-only, ordered JSON** — *is* the table. Delete it and you're back to a pile of Parquet.

A commit is a set of actions

00000.json
v0 · protocol
metaData
+add f1

→

00001.json
v1 · +add f2
–remove f1

→

00002.json
v2 · +add f3

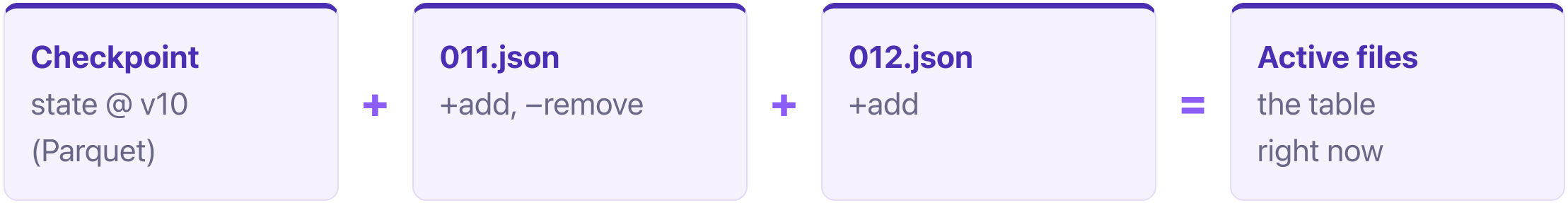
Each JSON line is an *action*

- `protocol` — reader/writer version
- `metaData` — schema, partitioning
- `add` / `remove` — a data file joins / leaves the table

Versions fall out for free

- Commit **N** = table **version N**
- The file set = **all `add`s minus all `remove`s** up to N
- An update = **remove old file + add new file**, one atomic commit

Reading = replay the log



- A reader loads the **latest checkpoint**, then replays the few JSONs after it → the exact set of live Parquet files.
- **Checkpoints every 10 commits** keep replay cheap — no walking thousands of tiny JSONs.
- **Time travel** is just: *stop replaying at version k*.

A checkpoint is a Parquet summary of the whole log up to that point — the same format, reused.

ACID via optimistic concurrency

How a write commits

1. **Read** the current version (snapshot)
2. **Write** new Parquet files (nobody sees them yet)
3. **Commit** = atomically create the next `NNN.json`

Why it's safe

- Creating `005.json` **fails if it already exists** → the other writer won for this version
- The loser **re-checks & retries** on top of the new state
- Readers only ever see **complete** commits → never a torn write

> ACID on cheap object storage, with **no lock server** — the ordered log filename *is* the lock.

bighub

Part 3

The operations

Your guides to applied AI

Append · overwrite · upsert

The three write modes

- **append** — add rows (a new `add` action)
- **overwrite** — replace the table's files (old ones become `remove`, still time-travellable)
- **upsert (MERGE)** — update some rows, insert others, in **one** commit

Our Silver customers table

- Bronze lands **raw** customer rows
- Silver keeps **one current row per customer**
- A change feed → **MERGE**: update the changed, insert the new

```
# write_delta — the whole table as a Delta table (mode: "overwrite" | "append")
write_deltalake(str(path), df.to_arrow(), mode=mode)
```

MERGE: the upsert

One statement: **update** rows that match the key, **insert** the ones that don't — atomically.

```
# upsert_delta(path, df, key="customer_id")
dt = DeltaTable(str(path))
(
    dt.merge(
        source=df.to_arrow(),
        predicate=f"t.{key} = s.{key}",   # match target t to source s on the key
        source_alias="s", target_alias="t",
    )
    .when_matched_update_all()            # existing customer → update
    .when_not_matched_insert_all()        # brand-new customer → insert
    .execute()
)
```

Matched rows are rewritten (remove + add), new rows appended — all in one versioned commit

Time travel

Read an older version

```
dt = DeltaTable(str(path))
if version is not None:
    dt.load_as_version(version)   # e.g. 0
return pl.from_arrow(dt.to_pyarrow_table())
```

Why it matters

- **Audit** — "what did Silver look like on Monday?"
- **Reproducibility** — pin a report to a version
- **Rollback** — restore a good state after a bad load
- **Debugging** — diff two versions

The old files are still referenced by old commits → the past is **queryable**, not gone.

Schema evolution

> A new column shows up in the feed. A plain append would **reject** the mismatch — Delta can evolve the schema in one atomic commit.

```
# append_with_evolution — add rows that introduce a NEW column
write_deltalake(str(path), df.to_arrow(), mode="append", schema_mode="merge")
```

- Old rows get **NULL** for the new column; new rows carry it.
- The change is recorded as a new `metaData` action → **versioned**, like everything else.
- Still **safe**: widening (add column, relax nullability) is allowed; a breaking change is rejected.

MERGE needs source & target schemas to line up — mind Polars dtypes (an int literal is Int32; cast to the column's type before concat).

Housekeeping: OPTIMIZE & VACUUM

OPTIMIZE

- Many small files → **compact** into fewer, bigger ones
- Faster reads, cheaper metadata
- (Databricks) **Z-order / liquid clustering** co-locate related rows

VACUUM

- Physically **delete files** no live version references
- Reclaims storage from old overwrites/merges
- **Retention (default 7 days)** — deletes break time travel **before** it

> Tension to teach: **time travel keeps old files; VACUUM removes them.** Retention is the dial between history and cost.

bighub

Part 4

The table-format landscape

Your guides to applied AI

Delta isn't the only answer

Delta Lake
Databricks
single JSON log
simple, ubiquitous

Iceberg
Netflix / Apache
snapshots + manifests
multi-engine first

Hudi
Uber
record-level index
high-frequency upserts

- Same promise: **ACID + time travel + schema evolution** over Parquet.
- **Iceberg** is today's common pick when no single vendor owns the stack (Spark, Trino, Flink, Snowflake read it natively).
- **Hudi** optimizes streaming-first, upsert-heavy ingestion — at more write-side complexity.

Different logs, one idea: metadata that says which files are the table

The catalog problem

> Every table format needs an answer to: *"what is the current version of table X, and where do its files live?"* — a concern **separate** from the format itself.

What a catalog does

- Maps **table name → location + current snapshot**
- Enforces one writer wins, governs access
- Lets many engines agree on "the table"

The options

- **Unity Catalog** (Databricks) — managed; Delta-first, now also Iceberg (UniForm)
- **Polaris · Nessie** — open, but a **service to run**
- ...that's infrastructure we deliberately skip locally

DuckLake: the catalog *is* a database

> Keep the catalog — table list, schema, snapshots — as **rows in an ordinary SQL database**; data still lands as Parquet next to it.

```
con.execute("INSTALL ducklake"); con.execute("INSTALL sqlite")
con.execute(f"ATTACH 'ducklake:sqlite:{catalog_path}' AS lake (DATA_PATH '{data_path}')")
con.execute("USE lake")
# time travel — read a past snapshot of the whole catalog
con.execute(f"SELECT * FROM customers AT (VERSION => {v0_snapshot})")
```

- Same guarantees (ACID, time travel, schema evolution), **zero extra infrastructure**.
- Open the `.sqlite` → `ducklake_table`, `ducklake_snapshot` … **just rows in a DB you already query**.

Gotcha: DuckLake versions the **whole catalog**, not one table — snapshot 0 is the `ATTACH` itself.

Interesting: how modern Delta got faster

Deletion vectors (Delta 2.3, 2023)

- A delete/update used to **rewrite whole files**
- Now: mark deleted rows in a **tiny side file** (merge-on-read)
- Delete 10 rows without rewriting a 500 MB file

Liquid clustering (Delta 3.x)

- Replaces rigid **partitioning + Z-order**
- **Redefine** clustering keys **without rewriting** the data
- Layout evolves with your query patterns

> The log protocol is **versioned** (`protocol` action) exactly so new tricks like these can land without breaking old readers.

bighub

Hands-on

Build the first Silver Delta table

Your guides to applied AI

Run it

```
git pull && uv sync
uv run eshop datagen && uv run eshop ingest   # if you don't have Bronze yet
uv run eshop delta-demo                       # initial load → upsert → time travel → evolve
```

Then look at what makes it a table:

```
ls data/lake/silver/customers_delta/_delta_log/   # the JSON commits + checkpoint
```

> No JVM, no cluster — `deltalake` (delta-rs) runs the whole protocol locally in Python.

Today's exercise

Fill in the `# TODO`s in `src/eshop/silver/delta_io.py`:

1. `write_delta` — write a Polars frame as a Delta table with the given **mode**.
2. `upsert_delta` — **MERGE** a change set on a key (update matched, insert new).
3. `read_delta` — **time-travel** to a historical version when `version` is given.
4. **(harder)** `append_with_evolution` — append rows that add a new column (`schema_mode="merge"`).

Bonus: `src/eshop/silver/ducklake_io.py` — the same demo over **DuckLake + SQLite** (`uv run eshop ducklake-demo`).

The key lines

```
# 1 — write (overwrite | append)
write_deltalake(str(path), df.to_arrow(), mode=mode)
# 2 — MERGE upsert
dt.merge(source=df.to_arrow(), predicate=f"t.{key} = s.{key}",
         source_alias="s", target_alias="t") \
  .when_matched_update_all().when_not_matched_insert_all().execute()
# 3 — time travel
dt.load_as_version(version)
# 4 — schema evolution
write_deltalake(str(path), df.to_arrow(), mode="append", schema_mode="merge")
```

Deliverable: a `customers_delta/` table with ≥ 2 versions; the upsert updates some + inserts one; version 0 still reads (time travel).

Key takeaways

- A **folder of Parquet is not a table** — no ACID, no upsert, no history, no safe concurrent writes.
- **Delta = Parquet + an ordered** `_delta_log`; commits of `add` / `remove` actions, **checkpoints every 10**.
- **ACID via optimistic concurrency** — the next commit's filename is the lock; readers see only complete versions.
- **MERGE** upsert, **time travel** (`load_as_version`), and **schema evolution** are all just versioned commits.
- Delta is one of three (**Iceberg · Hudi**); every format needs a **catalog** — or skip the service with **DuckLake**.

Next time

Session 05 — Silver: cleaning, data contracts and quality

- Turn raw Bronze into **trustworthy Silver**: typing, deduplication, canonical rows
- **Data contracts** & validation — don't let bad data in
- Quarantine, quality metrics, and failing a load on purpose

References & further reading

- **Delta Lake transaction log protocol** (actions, checkpoints) — github.com/delta-io/delta · PROTOCOL.md
- Unpacking the transaction log (Databricks) — databricks.com/blog
- delta-rs (`deltalake` Python, no JVM) — delta-io.github.io/delta-rs
- Deletion vectors & liquid clustering — docs.databricks.com
- Apache Iceberg — iceberg.apache.org · Apache Hudi — hudi.apache.org
- DuckLake — catalog in a SQL database — ducklake.select

bighub

Thank you

Let's build.

www.bighub.ai

Your guides to applied AI